

Dijkstra's Algorithm

A line-by-line trace through five west coast cities

What this guide covers: Dijkstra's shortest-path algorithm, explained by tracing the exact Python code as it runs on a small graph of west coast cities. Every iteration shows which line of code is executing and how the variables change.

The problem

You're standing in Seattle. You want to know the shortest driving time to every other city — Portland, San Francisco, Las Vegas, and Phoenix. The roads have different travel times, and some routes go through other cities. You want the algorithm to figure out the best route to each destination automatically.

The road map

Edge	Hours
Seattle ↔ Portland	3
Portland ↔ San Francisco	10
Portland ↔ Las Vegas	20
San Francisco ↔ Las Vegas	9
San Francisco ↔ Phoenix	11
Las Vegas ↔ Phoenix	5

Notice: Las Vegas can be reached two ways from Seattle. Directly via Portland ($3+20 = 23$ hours) or through San Francisco ($3+10+9 = 22$ hours). Dijkstra figures out which is shorter automatically. That comparison is the climax of the trace.

The analogy: Dij the traveler

Imagine a traveler named Dij who has three magic items:

Magic item	What it is in code	What it does
Notebook	<code>dist</code> (dict)	One page per city, where Dij writes the best-known travel time. Pages start

To-do list	queue (min-heap)	Always hands back the ticket with the smallest distance first.
DONE stamp	settled (set)	Once a city is stamped DONE, Dij never touches it again.

The lifecycle of every city: starts as unvisited ($\text{dist} = \infty$), gets pushed into the queue when a neighbor relaxes it, eventually pops out of the queue and gets stamped DONE. Once DONE, its dist is final.

The code we will trace

The trace below walks through every line as it executes. Refer back to this code throughout.

```
import heapq

def dijkstra(graph, start):
    dist = {city: float('inf') for city in graph} # line 1
    dist[start] = 0 # line 2
    queue = [(0, start)] # line 3
    settled = set() # line 4

    while queue: # line 5
        d, city = heapq.heappop(queue) # line 6
        if city in settled: # line 7
            continue # line 8
        settled.add(city) # line 9

        for neighbor, weight in graph[city]: # line 10
            new_dist = d + weight # line 11
            if new_dist < dist[neighbor]: # line 12
                dist[neighbor] = new_dist # line 13
                heapq.heappush(queue, (new_dist, neighbor)) # line 14

    return dist # line 15
```

What each line is doing, in plain English

- Lines 1-2** Initialize the notebook. Everyone is at ∞ except the start (0).
- Lines 3-4** Seed the queue with the start city. Empty out the DONE list.
- Line 5** While there's still work to do (queue not empty).
- Line 6** Pop the closest unsettled city. This is the heart of Dijkstra.
- Lines 7-8** If we already finished this city (stale entry), throw it out.
- Line 9** Mark this city as DONE. Its distance is now final.
- Line 10** Loop through this city's neighbors.
- Line 11** How far would the neighbor be if we went through here?
- Line 12** Is that better than what we had? This is RELAXATION.
- Lines 13-14** If yes, update the notebook and push the neighbor.
- Line 15** Return the final distances.

The trace, step by step

Each step shows the code lines executing, what's happening, and the state after.

STEP 0 Initialize the notebook and queue lines 1-4

Five cities. Notebook starts with ∞ everywhere except Seattle (0). Queue gets one ticket. Settled is empty.

city	dist	queue (sorted)	settled
Seattle	0	(0, Seattle)	(none)
Portland	∞		
SF	∞		
Vegas	∞		
Phoenix	∞		

STEP 1 Pop Seattle, relax its neighbors lines 5-14

Line 5: queue not empty, enter loop. **Line 6:** pop (0, Seattle); now $d=0$, city=Seattle. **Line 7:** Seattle in settled? No. **Line 9:** add Seattle to settled. **Line 10:** Seattle's only neighbor is Portland.

Inner loop, neighbor=Portland, weight=3. **Line 11:** $\text{new_dist} = 0 + 3 = 3$. **Line 12:** Is $3 < \infty$? Yes. **Line 13:** $\text{dist}[\text{Portland}] = 3$. **Line 14:** push (3, Portland).

city	dist	queue (sorted)	settled
Seattle	0	(3, Portland)	Seattle
Portland	3		
SF	∞		
Vegas	∞		
Phoenix	∞		

STEP 2 Pop Portland, relax three neighbors lines 5-14

Line 6: pop (3, Portland); $d=3$. **Line 9:** settle Portland. **Line 10:** Portland's neighbors are Seattle, SF, Vegas. Inner loop runs three times.

Seattle (weight 3): $\text{new_dist} = 6$. Is $6 < 0$? **No** $\rightarrow$ skip.

SF (weight 10): $\text{new_dist} = 13$. Is $13 < \infty$? **Yes** $\rightarrow$ update, push (13, SF).

Vegas (weight 20): $\text{new_dist} = 23$. Is $23 < \infty$? **Yes** $\rightarrow$ update, push (23, Vegas).

city	dist	queue (sorted)	settled
Seattle	0	(13, SF)	Seattle
Portland	3	(23, Vegas)	Portland
SF	13		
Vegas	23		
Phoenix	∞		

STEP 3 Pop SF — the better-route discovery lines 5-14

Line 6: pop (13, SF); d=13. **Line 9:** settle SF. **Line 10:** SF's neighbors are Portland, Vegas, Phoenix.

Portland (weight 10): new_dist = 23. Is $23 < 3$? **No** → skip.

Vegas (weight 9): new_dist = $13 + 9 = 22$. Is $22 < 23$? **YES — BETTER ROUTE!** Update dist[Vegas] from 23 to 22. Push (22, Vegas) onto the queue. The old (23, Vegas) is still sitting in the queue — we don't remove it. It's now stale.

Phoenix (weight 11): new_dist = 24. Is $24 < \infty$? **Yes** → **update, push (24, Phoenix).**

city	dist	queue (sorted)	settled
Seattle	0	(22, Vegas)	Seattle
Portland	3	(23, Vegas) (stale)	Portland
SF	13	(24, Phoenix)	SF
Vegas	22		
Phoenix	24		

The climax. dist[Vegas] just dropped from 23 to 22 because SF offered a shorter route. This is what relaxation does: when we find a better path through the current city, we update the neighbor's distance and push a fresh ticket. The old ticket (23, Vegas) becomes stale — we'll deal with it later when it pops.

STEP 4 Pop Vegas, no improvements lines 5-14

Line 6: pop (22, Vegas); d=22. **Line 9:** settle Vegas. **Line 10:** Vegas's neighbors are Portland, SF, Phoenix.

Portland (weight 20): new_dist = 42. Is $42 < 3$? No → skip.

SF (weight 9): new_dist = 31. Is $31 < 13$? No → skip.

Phoenix (weight 5): new_dist = $22 + 5 = 27$. Is $27 < 24$? **No** → **skip**. Going via Vegas is worse.

Nothing changes this iteration. The relaxation check on line 12 quietly rejects all three.

city	dist	queue (sorted)	settled
Seattle	0	(23, Vegas) (stale)	Seattle
Portland	3	(24, Phoenix)	Portland
SF	13		SF
Vegas	22		Vegas
Phoenix	24		

STEP 5 The stale entry skip lines 5-8

Line 6: pop (23, Vegas); d=23. **Line 7:** Is Vegas in settled? **YES — it was settled in iteration 4!**
Line 8: continue. Throw the ticket away without processing. Lines 9–14 are skipped entirely.

city	dist	queue (sorted)	settled
Seattle	0	(24, Phoenix)	Seattle
Portland	3		Portland
SF	13		SF
Vegas	22		Vegas
Phoenix	24		

This is why settled exists. When iteration 3 found a better route to Vegas, we didn't bother removing the stale (23, Vegas) — removing arbitrary entries from a heap is expensive. We let it pop later and skip it then. The check on line 7 is the bouncer at the door.

STEP 6 Pop Phoenix, the last city lines 5–14

Line 6: pop (24, Phoenix); d=24. **Line 9:** settle Phoenix. **Line 10:** Phoenix's neighbors are SF, Vegas. Both already settled. Both relaxation checks fail (35 not less than 13, 29 not less than 22). Nothing changes.

city	dist	queue (sorted)	settled
Seattle	0	(empty)	Seattle
Portland	3		Portland
SF	13		SF
Vegas	22		Vegas
Phoenix	24		Phoenix

STEP 7 Loop exits, return dist lines 5, 15

Line 5: queue is empty. The while loop condition is false. Fall through to **line 15:** return dist. The algorithm terminates.

The final answer

Destination	Hours	Route
Seattle	0	(starting point)
Portland	3	Seattle → Portland

San Francisco	13	Seattle → Portland → SF
Las Vegas	22	Seattle → Portland → SF → Vegas
Phoenix	24	Seattle → Portland → SF → Phoenix

Two interesting results. Vegas is reached through SF (22 hours), not directly via Portland (which would have been 23). And Phoenix is reached through SF (24), not via Vegas (which would have been 27). Dijkstra found these optimal routes automatically by always processing the closest unsettled city.

Why settling is safe (the key insight)

When we pop a city from the queue and settle it, we claim its distance is final. Future iterations won't find a shorter path. Why can we be sure?

Suppose we just popped Portland with $\text{dist} = 3$. Could there be a shorter path to Portland we haven't found yet? It would have to go through some other city X first.

If X is already settled, its dist is final and we already considered the route through it during relaxation. If X is unsettled, then X has $\text{dist} \geq 3$ (because Portland was the smallest item in the queue). Any path through X to Portland is at least $\text{dist}[X] + \text{edge_weight} \geq 3 + 0 = 3$. **Edges only add to the total — they never subtract.** So no shorter path exists.

This guarantee depends entirely on non-negative edge weights. If edges could be negative, an unsettled city could 'reach back' with a shortcut that decreases the total. Dijkstra would settle cities prematurely. For graphs with negative edges, use Bellman-Ford instead.

Confusions resolved (the FAQ)

When we pop Seattle, do we update all reachable cities?

No — only Seattle's **direct neighbors**. In our graph, Seattle's only neighbor is Portland, so we update only Portland. SF and Vegas are not touched yet — they'll be reached when we pop Portland in the next iteration. Dijkstra spreads outward one edge at a time.

Why does the queue contain duplicate entries for Vegas?

When iteration 3 found a shorter route to Vegas (22 vs 23), we pushed a new ticket (22, Vegas) but didn't remove the old (23, Vegas). Removing an arbitrary entry from a heap is expensive — $O(n)$. It's cheaper to leave stale entries and skip them when they pop. The check on line 7 catches them.

What if a shorter path to a settled city is discovered later?

It can't be. The math guarantees that when a city pops from the queue, no shorter path exists or can be found later. This relies on edges being non-negative. With negative edges, the guarantee breaks and you need Bellman-Ford.

What does the settled set actually do?

It guards against re-processing cities. Without it, when a stale entry pops (like (23, Vegas) in iteration 5), we'd run lines 9–14 again, doing redundant work and possibly making bad updates. The settled set is the bouncer that lets each city be processed exactly once.

Why does dist only ever decrease?

Line 12 is the gatekeeper: `if new_dist < dist[neighbor]`. We only update when the new value is smaller. Once a city is settled, no further updates can succeed for it, because any `new_dist` computed for a settled city would be $\geq$ its current value.

Can the same city appear in the queue more than twice?

Yes, in graphs with more paths. Each time relaxation succeeds, a new ticket gets pushed. All but one of them will be stale by the time they pop. The skip-if-settled check handles all of them uniformly.

The two-minute recap

If you remember nothing else, remember this:

The data structures: three things working together. **dist** is the notebook of best-known distances. **queue** is a min-heap that always hands back the smallest. **settled** is the set of cities we're done with.

The loop: pop the smallest unsettled city → if it's stale (already settled), skip → otherwise settle it → relax all its neighbors → repeat until the queue is empty.

Relaxation: for each neighbor, compute $\text{new_dist} = \text{current_dist} + \text{edge_weight}$. If that's smaller than the neighbor's notebook entry, update the entry and push the neighbor onto the queue with its new distance. Old tickets become stale but stay in the queue.

Why it works: when you pop a city, no shorter path exists. Other unsettled cities have larger or equal dist, and edges only add to the total. So the popped distance is final. **This requires non-negative edge weights.**

Where you'll see it

GPS routing (Google Maps, Waze) — finding the shortest route from where you are to where you're going.

Network routing (OSPF protocol) — packets finding the shortest path through routers.

Game pathfinding — game characters navigating around obstacles (usually A*, which is Dijkstra plus a heuristic).

Robotics — robots planning a path through a warehouse or environment.

Built around the trace: Seattle → Portland → SF → Vegas → Phoenix. The west coast cities you asked for, run through the exact code line by line.